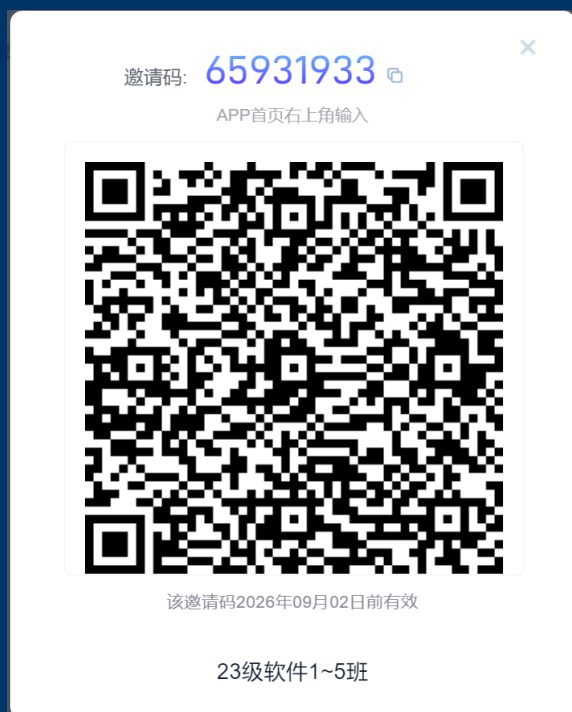
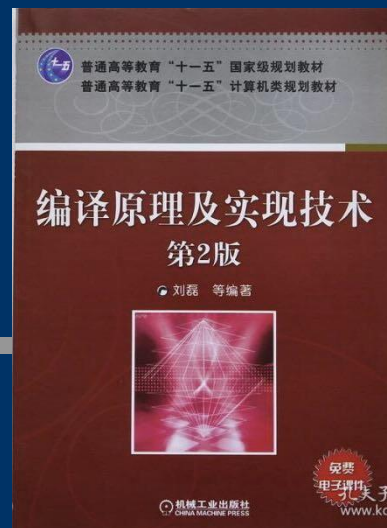


编译原理及实现

主讲：张 晶



课程说明



- ◆ 教材: 刘磊 《编译原理及实现技术》 (第2版)
- ◆ 学习和考核方式:

40学时理论
(1~11周, 第5周空)

超星MOOC (视频, 作业, 讨论, 活动)

[20%]

期末考试 [70%]

12学时上机 [10%]

(12~14周)

教学内容安排

教材

- ◆ 第1章 编译引论
- ◆ 第2章 形式语言与自动机理论
- ◆ 第3章 词法分析
- ◆ 第4章 语法分析-自顶向下分析方法
- ◆ 第5章 语法分析-自底向上分析方法
- ◆ 第6章 语义分析和符号表
- ◆ 第7章 中间代码生成
- ◆ 第8章 中间代码优化
- ◆ 第9章 运行时存储空间的组织和管理
- ◆ 第10章 目标代码生成

授课

- ◆ 第1章 编译引论
- ◆ 第2章 词法分析(自动机理论)
- ◆ 第3章 语法分析-形式语言基础
- ◆ 第4章 语法分析-自顶向下分析方法
- ◆ 第5章 语法分析-自底向上分析方法(了解基本分析思想)
- ◆ 第6章 语义分析和符号表
- ◆ 第7章 中间代码生成
- ◆ 第8章 中间代码优化
- ◆ 第9章 运行时存储空间的组织和管理
- ◆ 第10章 目标代码生成

1. 什么是编译程序

- ◆ 编译程序，也叫编译器，是一种具有翻译功能的程序，其作用是将某种高级语言编写的程序翻译成语义等价的低级语言程序。



- 高级语言程序必须经过翻译才能执行吗？
- 编译是翻译的唯一途径吗？
- 编译程序是怎么翻译的呢？
- 编译程序是怎么开发出来的？
- 为什么要学习编译原理课程？

1. 什么是编译程序

- ◆ 程序设计语言的发展
- ◆ 高级语言的实现

1.1 程序设计语言的发展

■ 第一代：机器语言

- 00000000001000100011000000100000
- 机器指令=操作码+操作数地址，计算机可直接执行

■ 第二代：汇编语言

- STO id2 R2
- LDA c R1
- ADDF id3 R1
- 汇编指令=符号化的机器指令，计算机不能直接执行，需要用汇编程序汇编成机器语言程序后才可运行

■ 第三代：高级语言

- $x = f(i,j)$
- 计算机不能直接执行高级语言写的程序，需要用“翻译程序”翻译成机器语言程序后才可运行

1.1 程序设计语言的发展

- 程序设计语言的分类:
- 功能:
 - 科学计算(Fortran); 商业数据处理(cobol); 符号计算(Lisp); 格式处理(Latex); 逻辑处理(Prolog);脚本语言(Perl, JavaScript);数据库处理(SQL, JQuery);数据统计和分析(R,MATLAB);图形图像处理(PhotoShop,Flash);多功能(C,C++,C#,Java,VB, Python)
- 抽象级别:
 - 低级语言: 机器语言、汇编语言
 - 高级语言: Ada, Basic, C, C++, C#, Dephi, Fortran, ML, Lisp, SmallTalk, Pascal, Ruby,
- 编程机制:
 - 过程式、函数式、对象式、逻辑式

◆ 过程式：

```
#define TYPE int
void square(TYPE x[], int n)
{
    for(int i=0; i<n; i++)
        *(x+i)*=*(x+i);
}
```

典型的过程式语言： Basic、Pascal、C等

◆ 对象式： C++， Java， python等

◆ 函数式：

```
fun    square([])=[]
| square(a::x)=a*a  :: square(x)
```

典型的函数式语言： Lisp、ML、Haskell、F#等

高级语言的不同范式

◆ 逻辑式: Prolog

【事实】

Jack 喜欢 Susan
John 喜欢 Mary
Tom 喜欢 Cathy
Mark 喜欢 Ellen
Tom 喜欢 Ellen

【规则】

如果Tom喜欢X，那么Jack喜欢X

domains

person, another = symbol

predicates

likes(person, another).

clauses

likes(jack, susan).

likes(john, mary).

likes(tom, cathy).

likes(mark, ellen).

likes(bob, tom).

likes(richard, ellen).

likes(tom, ellen).

likes(jack, X) if likes(tom, X).

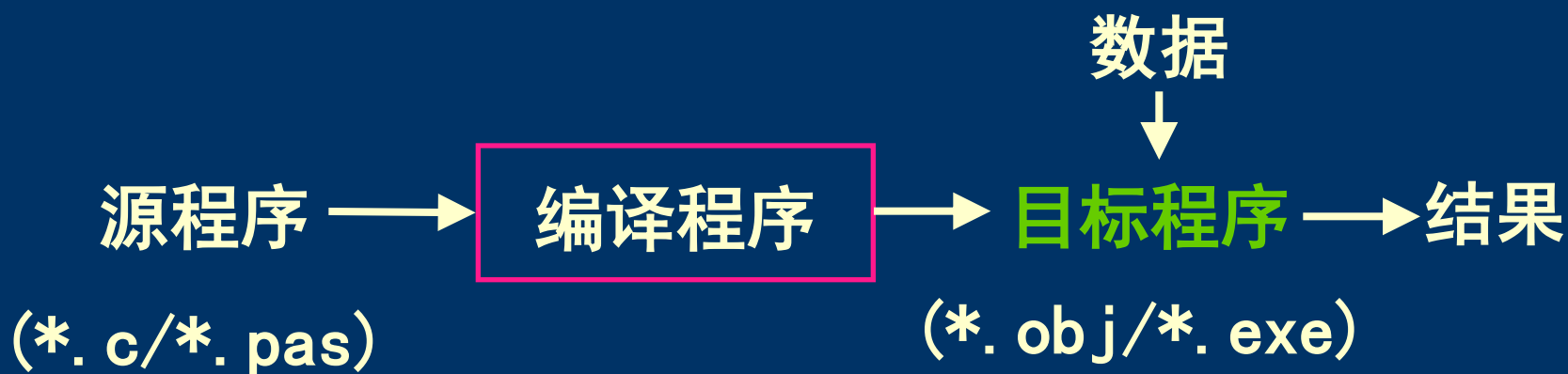
likes(jack, ellen)?

1.2 高级语言的实现

- 尽管用汇编语言和高级语言编写程序比用机器语言方便得多，但计算机只能识别机器语言写成的机器指令，因此必须有这样一个程序，它能够把用汇编语言或高级语言编写的程序转换成语义等价的机器语言程序，执行这种转换功能的程序统称为翻译程序。
- 汇编语言的翻译程序称为汇编程序或汇编器(Assembler)。
- 高级语言的翻译程序称为编译程序(Compiler)或解释程序(Interpreter)，对应的实现方式分别为编译方式或解释方式。
- 除编译方式和解释方式外，高级语言偶尔也会采用转换的方式实现。

1.2 高级语言的实现

- ◆ **编译程序**：是将某种高级语言编写的程序翻译成语义等价的低级语言程序的程序。其中高级语言编写的程序称为“源程序”，低级语言程序称为“目标程序”。目标程序可以是机器语言程序或汇编语言程序。用于书写源程序的语言称为源语言，翻译得到的目标程序对应的语言称为目标语言。



编译过程是程序到程序的翻译过程

1.2 高级语言的实现

- ◆ **解释程序**：其作用是将源程序逐条语句地翻译成目标指令序列，并且是边翻译边执行，当翻译完成时，目标程序的执行也随之完成。可见，解释程序不会生成目标程序。



(*.bas/*.java/*.py)

解释过程是语句到执行结果的翻译过程

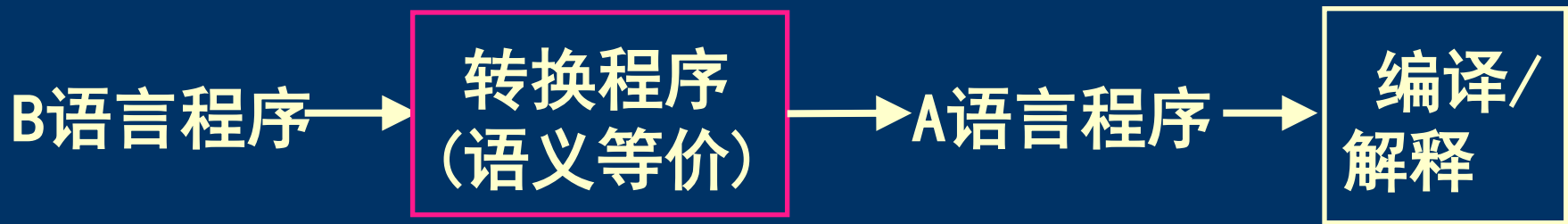
编译方式与解释方式的比较

- ◆ 相同点：二者实现技术相似，均需要经过词法、语法、语义、代码生成等步骤；
- ◆ 区别：（本质区别：是否生成目标程序！！！）
 - 实现机制：翻译（程序to程序）vs. 解释（语句to指令序列）；
 - 执行速度：解释器执行速度要慢；
 - 空间开销：解释器需要保存的信息较多，空间开销大；
 - 可移植性：解释方式具有更好的可移植性；
 - 基于解释执行的程序错误诊断效果好，可以动态修改；
 - 基于解释方式有利于人机交互；

综上，根据实际需求选择编译方式和解释方式，大多数的高级语言采用编译方式。

1.2 高级语言的实现

- ◆ **转换方式**：编译程序的开发代价是很昂贵的。在已有A语言的编译程序的前提下，若想为B语言开发编译程序，可首先考虑能否利用A语言的编译程序。即首先将B语言程序转换成等价的A语言程序，然后再利用A语言的编译程序将其翻译成目标程序。



- ◆ 转换方式的第二个前提是：B语言与A语言的近似度很高，或B语言是A语言的扩展。
- ◆ 转换方式的范例：C++语言的实现

2. 编译程序的逻辑结构

- ◆ 编译程序的翻译过程与自然语言的翻译过程类似：先理解原文再翻译成对等的目标语言文章。

- 你能够通过自己的努力实现你的梦想!

识别单词

检查语法

检查语义

生成译文

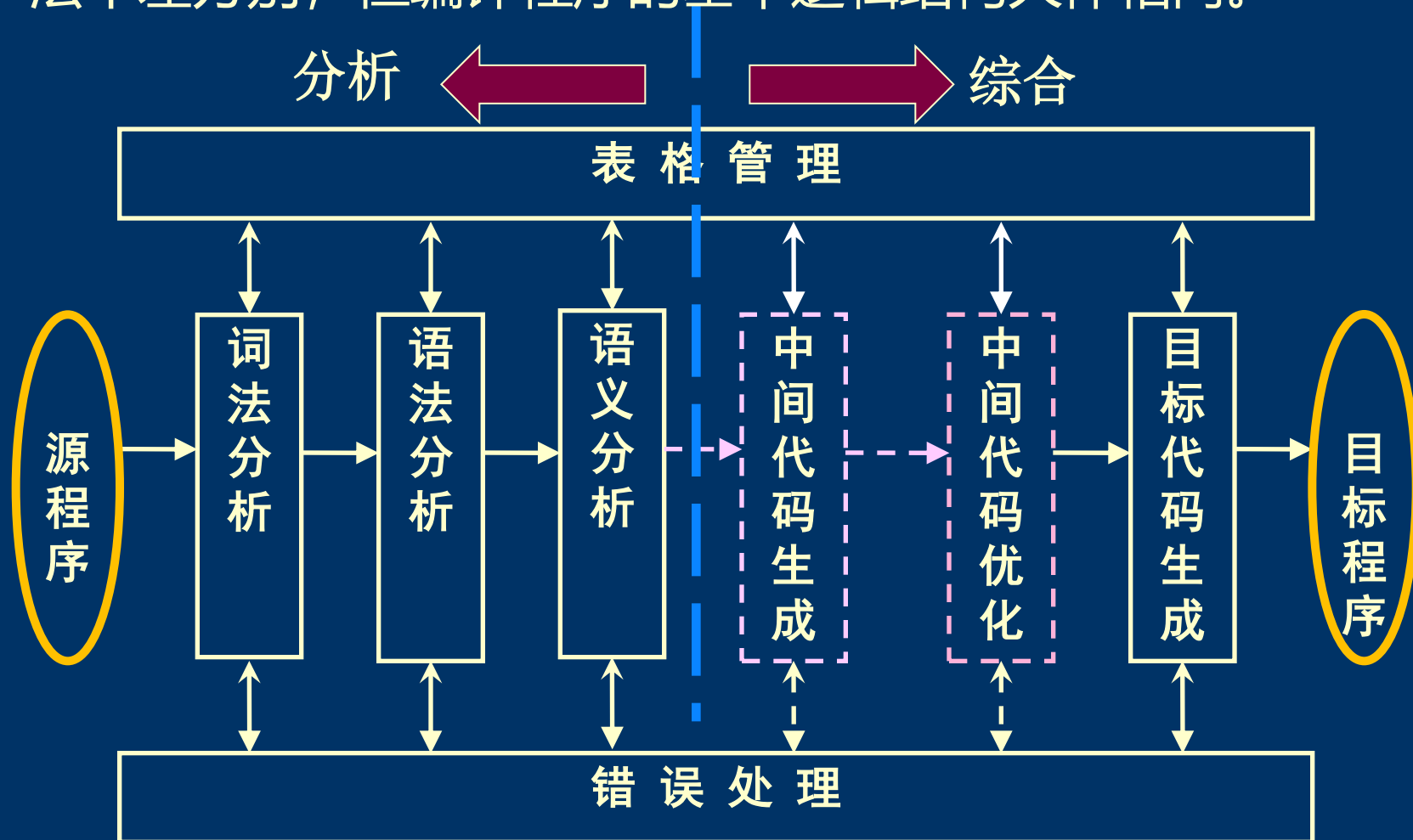
信达雅

- 你能够通过自己的努力实现你的梦想!

You can put your dreams into reality through your efforts!

2. 编译程序的逻辑结构

- 编译程序是复杂的系统软件，不同的编译程序其实现方法千差万别，但编译程序的基本逻辑结构大体相同。



2. 编译程序的逻辑结构

- ◆ 词法分析：识别由字符组成的高级语言程序中的单词，并将其转化成一种内部表示（Token）的形式，同时检查是否存在词法错误。

如有：x1=100；

机器视角：'x' '1' '=' '1' '0' '0' ';' ;

单词：符合某种规律的字符串

Token：一种数据结构，将单词信息存储起来供语法分析程序使用

词法错误：L@7

2. 编译程序的逻辑结构

- ◆ 语法分析：根据语言定义的语法规则来检验程序中是否存在语法错误。

每个语言都有自己的语法，编写程序一定要符合这个语言的语法约定，否则将不能编译执行。

C语言：

i++ 合法

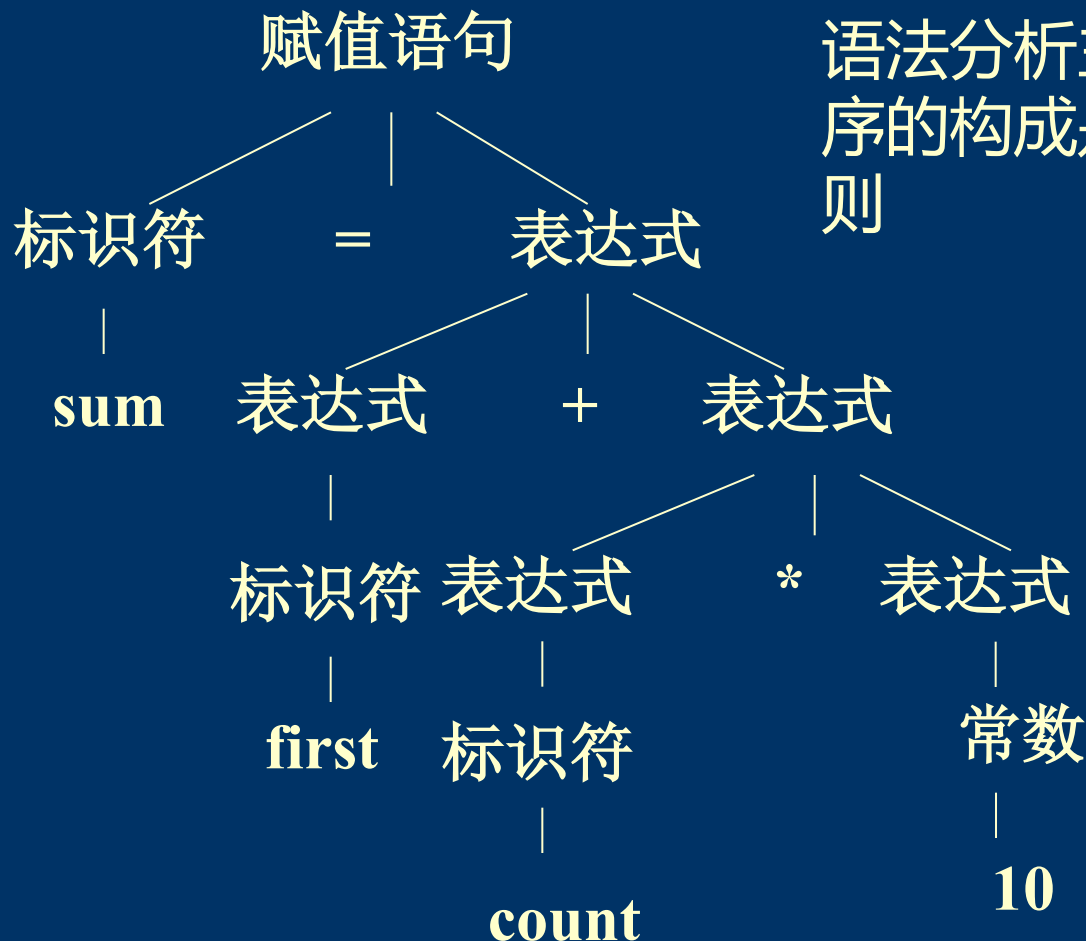
Pascal语言：

i := i + 1 合法

i++ 不合法

语法分析的示例

◆ `sum = first + count * 10`



语法分析主要用来检查程序的构成是否符合语法规则

2. 编译程序的逻辑结构

- ◆ 语义分析：检查源程序有无语义错误，为代码生成阶段收集类型信息。

语义分析的示例

Pascal语言程序片段：

VAR

 first: real;

 count: char;

BEGIN

 sum:=first + count * 10

END.

语义错误：

- 1、sum有使用而无定义；
- 2、count为字符类型变量不能进行乘法运算。

C语言程序片段：

float first;

char count;

sum=first + count * 10

语义错误：

- 1、sum有使用而无定义；

2. 编译程序的逻辑结构

- ◆ 中间代码生成：将源程序转换成一种称为中间代码的内部表示形式，便于优化和移植。
- ◆ 不是编译程序的必经阶段！
- ◆ 不考虑优化和移植时，可以不生成中间代码直接生成目标代码！

中间代码生成的示例

例:

```
VAR sum, first, count: real;  
BEGIN  
    sum:=first + count * 10  
END.
```

生成如下四元式形式的中间代码序列:

- 1、 (int-to-real, 10 , - ,t1)
- 2、 (*, count , t1 ,t2)
- 3、 (+, first , t2 ,t3)
- 4、 (:=, t3 ,- , sum)

2. 编译程序的逻辑结构

- ◆ 中间代码优化：变换或改造中间代码，使生成的目标代码更为高效，即节省时间和空间。和程序算法的高效无关，更多的是针对于程序具体运行时的内部优化，尤其针对有特殊要求的编译器。
- ◆ 不是编译程序的必经阶段，当对目标代码的运行效率要求较高时，才考虑进行中间代码的优化，优化要付出代价，优化时要考虑性价比。

中间代码优化的示例

例: `sum:=first + count * 10`

生成如下四元式形式的中间代码序列:

- 1、 `(int-to-real, 10, -, t1)`
- 2、 `(*, count, t1, t2)`
- 3、 `(+, first, t2, t3)`
- 4、 `(:=, t3, -, sum)`



生成如下优化后的四元式形式的中间代码序列:

- 2、 `(*, count, 10.0, t2)`
- 3、 `(+, first, t2, t3)`
- 4、 `(:=, t3, -, sum)`

2. 编译程序的逻辑结构

- ◆ 目标代码生成：将中间代码变换为特定机器上的机器指令代码或汇编指令代码。

例： `sum:=first + count * 10`

生成如下汇编代码：

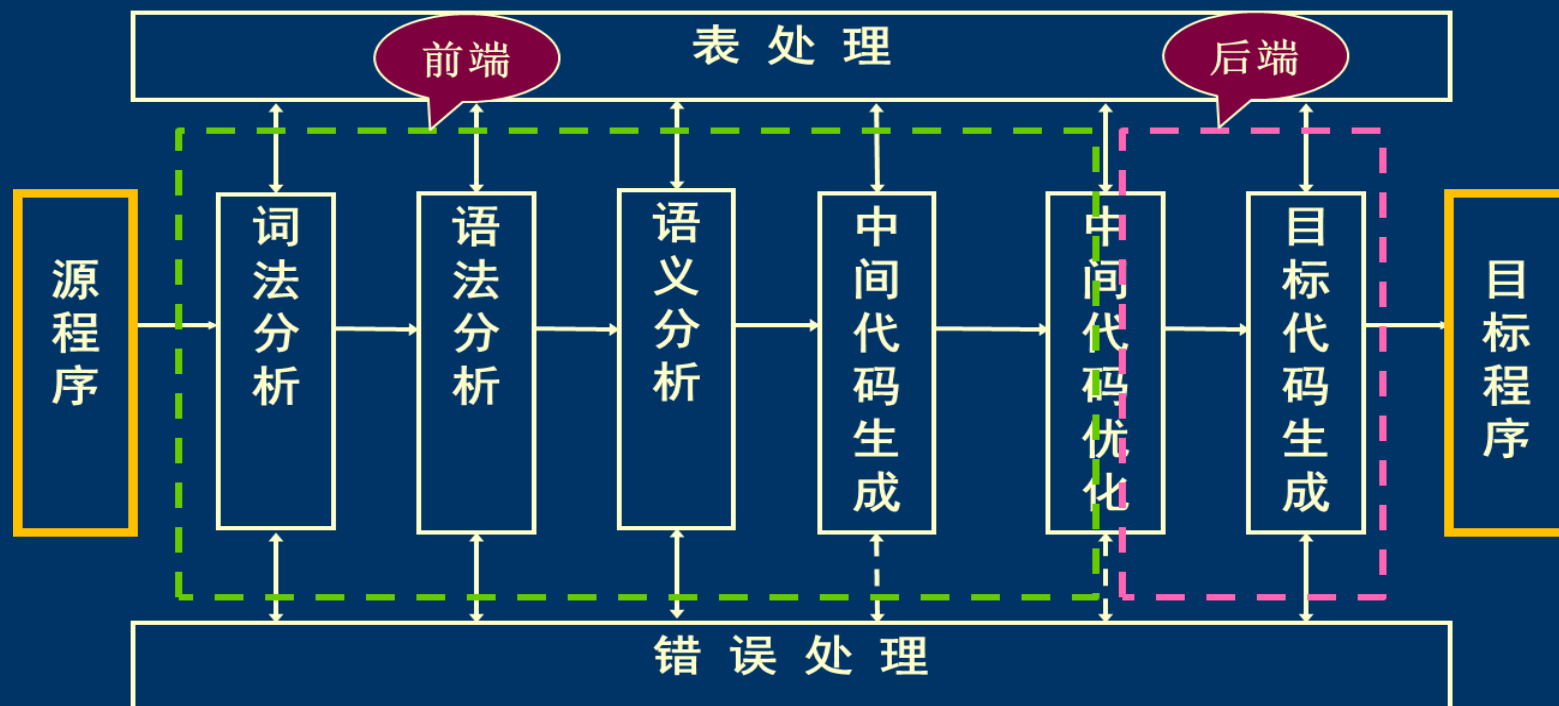
<code>(*, count, 10.0, t₂)</code>		1. MOV count, R1
<code>(+, first, t₂, t₃)</code>	➡	2. MULT R1, #10.0
<code>(:=, t₃, -, sum)</code>		3. MOV first, R2
		4. ADD R1, R2
		5. MOV R1, sum

2. 编译程序的逻辑结构

此外，编译过程还有两部分工作贯穿始终：

- ◆ 错误处理：当编译阶段有错误出现时，由相应的错误处理模块给出解决方案，使得编译器能够继续进行下去。
- ◆ 表格管理：为了合理的管理（构造、查找、更新……）表格（符号表、类型信息表……），设立一些专门子程序称为表格管理程序。

- ◆ **前端(front end)**: 编译程序中与源语言有关, 与目标程序无关的部分, 称为前端。
 - 通常包括词法分析、语法分析、语义分析、中间代码生成, 与目标机无关的中间代码优化部分。
- ◆ **后端(back end)**: 编译程序中与源语言无关, 与目标程序有关的部分, 称为后端。
 - 通常包括与目标机有关的中间代码优化、目标代码生成等部分。



3. 编译程序的开发

- ◆ 1. 用机器语言写编译程序
- ◆ 2. 自展法： L_0 是 L 的子集
 - 用机器语言写 L_0 的编译器
 - 扩展 L_0 到 L_1 ，用 L_0 写 L_1 的编译器
 - 扩展 L_1 到 $L_2, \dots$
 - $\dots$
 - 扩展 L_i 到 L ，用 L_i 写 L 的编译器

3. 编译程序的开发

- ◆ 3. 移植法：
 - 途径1：目标代码1转换成目标代码2
 - 途径2：保留前端，改写后端
 - 途径3：交叉编译
- ◆ 4. 转换法(预处理法)：C++
- ◆ 5. 自动生成(工具法)：利用Flex, Yacc, LLGen, Bison等工具自动生成词法分析器、语法分析器，从而减少编译程序开发的工作量

3. 编译程序的开发

- ◆ 设计中要注意的问题
 - a) 准确的理解源语言
 - b) 确定以及理解目标语言（汇编语言，机器语言，虚拟机指令）
 - c) 确定编译要求（是否优化，优化到什么程度）
 - d) 确定分遍
 - e) 具体设计

3. 编译程序的开发

◆ 编译程序的分遍

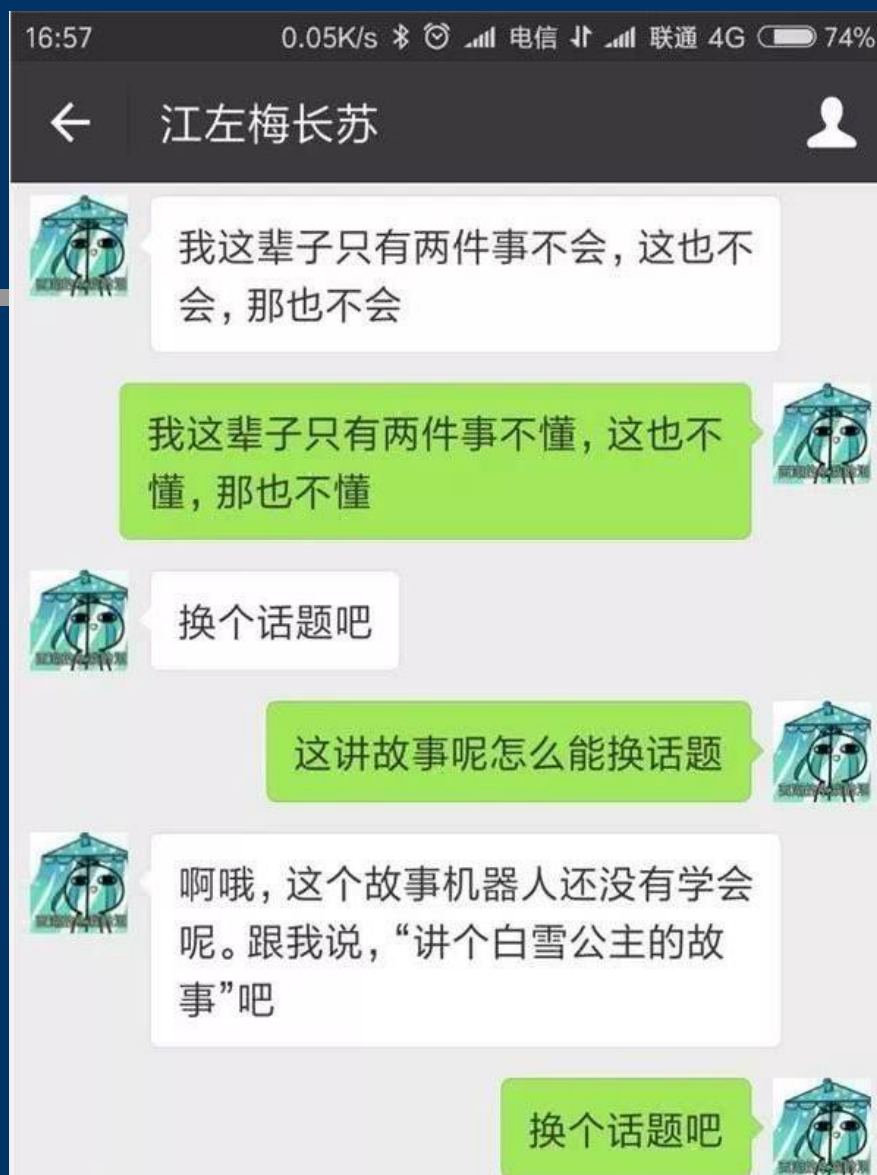
所谓“遍”就是对源程序或源程序的中间表示形式（如Token序列，语法树，四元式中间代码等）从头到尾扫描一次，并作加工处理，生成新的中间结果或目标程序；

分遍就是对源程序或源程序的中间表示形式从头到尾扫描几次。

- a. 分遍的理由
- b. 本课程的分遍策略

4. 课程的意义

- ◆ 加深对高级程序设计语言的理解和认识
- ◆ 是一种元级程序设计
- ◆ 提高程序（尤其是大型）的设计能力
- ◆ 可以培养抽象能力、形式化描述能力
- ◆ 编译技术可以应用到许多实际开发工作中



insert into vehicle(number_id,brand,makedate) **values**('豫HPU579','红旗',getdate())

insert into vehicle **values**('豫HPU33U','大众','银灰色',getdate())

insert into vehicle **values**('豫HPU123','红旗','黑色',getdate()-365*2)

delete vehicle **where** number_id = '豫HPU23U' --按条件删除

delete vehicle --删除表中全部记录

update vehicle **set** color = '红色' **where** brand = '红旗' --按条件更新车的颜色

update vehicle **set** makedate = getdate() --更新表中全部记录的出厂日期为当前时间

select number_id,makedate **from** vehicle **where** brand = '红旗' --按条件查找所需字段的数据

select top 2 * from vehicle --查询表中的前两条记录

select * from vehicle



AI同传

视频翻译



人工翻译

插件下载

APP下载

ssrsjzx

检测到中文



英语

翻译

人工翻译



比我差的人还没放弃，比我好的人仍在努力，我就更没资格说我无能为力！你配不上自己的野心，也辜负了曾经历的苦难。



People who are worse than me have not given up, people who are better than me are still working hard, and I am not qualified to say that I can do nothing! You are unworthy of your ambition and the suffering you have experienced.



报错

双语对照

